

6.4.6 PRU_ICSSG Broadside Accelerators

6.4.6.1 PRU_ICSSG Broadside Accelerators Overview

The PRU_ICSSG supports a broadside interface, which uses the XFR (XIN, XOUT, or XCHG) instruction to transfer the contents of PRUn, RTU_PRUn or TX_PRUn (where n = 0 or 1) registers to or from accelerators. This interface enables up to 31 registers (R0-R30, or 124 bytes) to be transferred in a single instruction. This section details the various accelerators that are available to the PRUn and/or RTU_PRUn and TX_PRUn through the broadside interface.

Each of those functions have a unique XIN ID to determine which operation will occur. For more information see [Table 6-403](#).

6.4.6.2 PRU_ICSSG Data Processing Accelerators Functional

6.4.6.2.1 PRU Multiplier with Accumulation (MPY/MAC)

This section describes the MAC (multiplier with accumulation) module integrated to PRU0/PRU1 cores, RTU_PRU0/RTU_PRU1 auxiliary cores and TX_PRU0/TX_PRU1 cores of PRU_ICSSGn (or CFG1, PRU1).

Each of the six PRU cores (PRU0/PRU1, RTU_PRU0/RTU_PRU1 auxiliary cores and TX_PRU0/TX_PRU1 transmit cores) has a designated unsigned multiplier with accumulation (MPY/MAC). The MAC supports two modes of operation: Multiply Only and Multiply and Accumulate.

The MAC is directly connected with the PRU internal registers R25-R29 and uses the broadside load/store PRU interface and XFR instructions to both control and mode of the MAC and import the multiplication results into the PRU.

The PRU MPY/MAC features are:

- Configurable Multiply Only and Multiply and Accumulate functionality via PRU register R25
- 32-bit operands with direct connection to PRU registers R28 and R29
- 64-bit result (with carry flag) with direct connection to PRU registers R26 and R27
- One clock cycle per operation
- PRU broadside interface and XFR instructions (XIN, XOUT) allow for importing multiplication results and initiating accumulate function
- Firmware can SEED the accumulator

6.4.6.2.1.1 PRU MAC Operations

6.4.6.2.1.1.1 PRU versus MAC Interface

The MAC directly connects with the PRU internal registers R25-R29 through use of the PRU broadside interface and XFR instructions. [Figure 6-201](#) shows the functionality of each register.

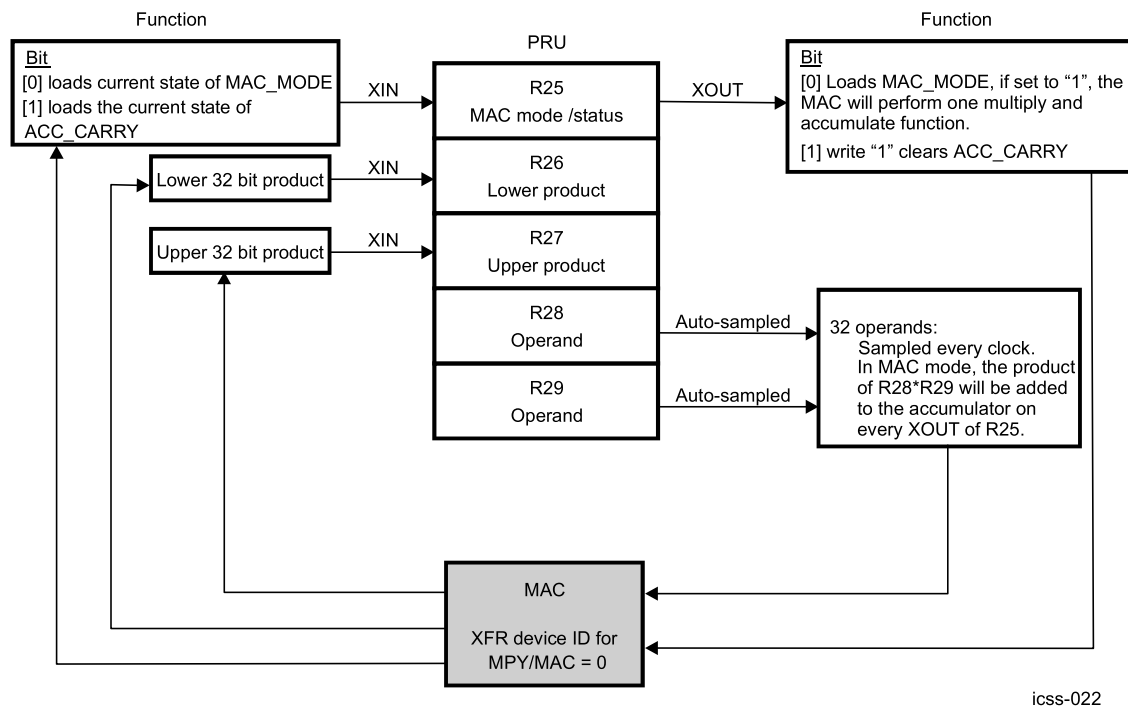


Figure 6-201. Integration of the PRU and MPY/MAC

The XFR instructions (XIN and XOUT) are used to load/store register contents between the PRU core and the MAC. These instructions define the start, size, direction of the operation, and device ID. The device ID number corresponding to the MPY/MAC is shown in [Table 6-427](#).

Table 6-427. MPY/MAC XFR ID

Device ID	Function
0	Selects MPY/MAC

The PRU register R25 is mapped to the MAC_CTRL_STATUS register ([Table 6-428](#)). The MAC's current status (MAC_MODE and ACC_CARRY states) is loaded into R25 using the XIN command on R25. The PRU sets the MAC's mode and clears the ACC_CARRY using the XOUT command on R25.

Table 6-428. MAC_CTRL_STATUS Register (R25) Field Descriptions

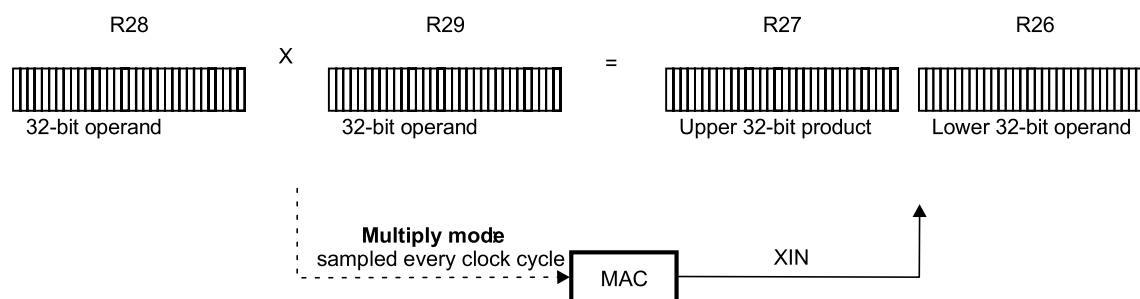
Bit	Field	Description
7-2	RESERVED	Reserved
1	ACC_CARRY	Write 1 to clear. It is sticky. It is set 0 cycles after the event. 0h: 64-bit accumulator carry has not occurred 1h: 64-bit accumulator carry occurred
0	MAC_MODE	0h: Accumulation mode disabled and accumulator is cleared 1h: Accumulation mode enabled

The two 32-bit operands for the multiplication are loaded into R28 and R29. These registers have a direction connection with the MAC. Therefore, XOUT is not required to load the MAC. In multiply mode, the MAC samples these registers every clock cycle. In multiply and accumulate mode, the MAC samples these registers every XOUT R25[7-0] transaction when MAC_MODE = 1.

The product from the MAC is linked to R26 (lower 32 bits) and R27 (upper 32 bits). The product is loaded into register R26 and R27 using XIN.

6.4.6.2.1.1.2 Multiply only mode(default state), MAC_MODE = 0

The [Figure 6-202](#) summarizes the MAC operation in "Multiply-only" mode, in which the MAC multiplies the contents of R28 and R29 on every clock cycle.



icss-023

Figure 6-202. MAC Multiply-only Mode- Functional Diagram

6.4.6.2.1.1.2.1 Programming PRU MAC in "Multiply-ONLY" mode

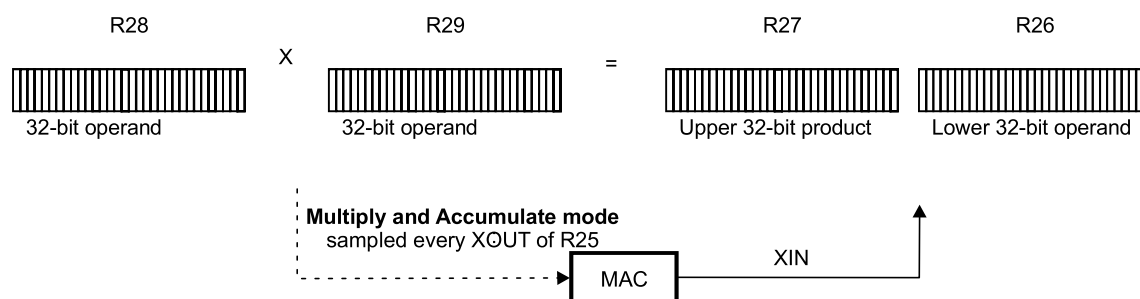
The following steps are performed by the PRU firmware for multiply-only mode:

1. 1. Enable multiply only MAC_MODE.
 - a. (a) Clear R25[0] for multiply only mode.
 - b. (b) Store MAC_MODE to MAC using XOUT instruction with the following parameters:
 - Device ID = 0
 - Base register = R25
 - Size = 1
2. 2. Load operands into R28 and R29.
3. 3. Delay at least 1 PRU cycle before executing XIN in step 4.
4. 4. Load product into PRU using XIN instruction on R26, R27.

Repeat steps 2 and 4 for each new operand.

6.4.6.2.1.1.3 Multiply and Accumulate Mode, MAC_MODE = 1

The [Figure 6-203](#) summarizes the MAC operation in "Multiply and Accumulate" mode. On every XOUT R25_REG[7-0] transaction, the MAC multiplies the contents of R28 and R29, adds the product to its accumulated result, and sets ACC_CARRY if an accumulation overflow occurs.



icss-024

Figure 6-203. MAC Multiply and Accumulate Mode Functional Diagram

6.4.6.2.1.1.3.1 Programming PRU MAC in Multiply and Accumulate Mode

The following steps are performed by the PRU firmware for multiply and accumulate mode:

1. Enable multiply and accumulate MAC_MODE.

(a) Set R25[1-0] = 1 for accumulate mode.

(b) Store MAC_MODE to MAC using XOUT instruction with the following parameters:

- Device ID = 0
- Base register = R25
- Size = 1

2. Clear accumulator and carry flag.

(a) Set R25[1-0] = 2 to clear carry flag (R25[1]=1) and clear accumulator (R25[0]=0).

(b) Store accumulator to MAC using XOUT instruction on R25.

3. Load operands into R28 and R29.

4. Multiply and accumulate, XOUT R25[1-0] = 1

Repeat step 4 for each multiply and accumulate using same operands.

Repeat step 3 and 4 for each multiply and accumulate for new operands.

5. Load the accumulated product into R26, R27, and the ACC_CARRY status into R25 using the XIN instruction.

1.

Firmware can load/seed the accumulator by doing XOUT of R27:R26. This will also clear the carry flag.

Note

Steps one and two are required to set the accumulator mode and clear the accumulator and carry flag.

6.4.6.2.2 PRU CRC16/32 Module

Each of the PRU0/PRU1 cores, RTU_PRU0/RTU_PRU1 auxiliary cores and TX_PRU0/TX_PRU1 transmit cores have a designated CRC16/32 module.

In general, CRC adds error detection capability to communication systems. The CRC encoder appends redundant bits (or CRC bits) to the systematic data message. During reception of the data message, the received data is also encoded with the same CRC encoder. The 2 sets of CRC bits are compared together. If they match, there were no transmission errors; and if they don't match, a transmission error has been detected.

CRC16/32 supports the following features:

- Supports CRC32:
 - $x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x+1$
- Supports CRC16:
 - $x^{16}+x^{15}+x^2+1$
- Supports CRC16 - CCITT:
 - $x^{16}+x^{12}+x^5+1$
- PRU broadside interface and XFR instructions (XIN, XOUT) allow for importing CRC results and executing accumulate function

6.4.6.2.2.1 PRU and CRC16/32 Interface

The CRC16/32 module directly connects with the PRU internal registers R25-R29 through use of the PRU broadside interface and XFR instructions. [Table 6-429](#) shows the functionality of each register.

The XFR instructions (XIN/XOUT/XCHG) are used to load/store register contents between the PRU core and the CRC16/32 module. These instructions define the start, size, direction of the operation, and device ID. The XFR device ID number corresponding to the CRC16/32 module is 1.

Table 6-429. CRC Register to PRU Port Mapping

CRC Register	R/W	Description	PRU Mapping
CRC_CFG	W	Always write all 4 bytes. bit [0] CRC32_ENABLE: 0: CRC16 mode is selected. Hardware will auto-set init state of CRC_SEED to 0000_0000h. However, for CRC16-CCITT software will need to write the init state of FFFF_FFFFh to CRC_SEED. Note: The CRC16 result value is only 16-bits. 1: CRC32 mode is selected. Hardware will auto-set init state of CRC_SEED will be FFFF_FFFFh. bit [1] CRC_32B_NOT_EMPTY: 0: CRC 32Byte buffer is empty 1: CRC 32Byte buffer is not empty bit [2] CRC16_MOD_ENABLE: 0: CRC16 ($x^{16}+x^{15}+x^2+1$) 1: CRC16-CCITT ($x^{16}+x^{12}+x^5+1$) - Note: CRC32_ENABLE field must = 0.	R25
CRC_DATA_8_BFLIP	R	8-bit flip of CRC_DATA. CRC_DATA_8_BFLIP has the same byte order as CRC_DATA[31-0], but each byte has all bits flipped. CRC_DATA_32_FLIP[7-0] = CRC_DATA[0-7] CRC_DATA_32_FLIP[15-8] = CRC_DATA[8-15] CRC_DATA_32_FLIP[23-16] = CRC_DATA[16-23] CRC_DATA_32_FLIP[31-24] = CRC_DATA[24-31] For CRC16, only CRC_DATA_8_BFLIP[15-0] are valid. No auto reset on CRC_DATA_8_BFLIP read.	R27
CRC_SEED	W	CRC SEED value. Hardware will auto-initialize the CRC_SEED value to 0000_0000h for CRC16 and FFFF_FFFFh for CRC32. Software only needs to initialize CRC_SEED if a different default value is required. For CRC16-CCITT, software needs to update initial CRC_SEED value to FFFF_FFFFh. Always write 4 bytes. Note: Reading the CRC_DATA register will reset the CRC value to the CRC_SEED state.	R28

Table 6-429. CRC Register to PRU Port Mapping (continued)

CRC Register	R/W	Description	PRU Mapping
CRC_DATA_32_BFLIP	R	Full 32-bit flip of CRC_DATA CRC_DATA_32_BFLIP[0] = CRC_DATA[31] ... CRC_DATA_32_BFLIP[31] = CRC_DATA[0] For CRC16, only CRC_DATA_32_BFLIP[31-16] are valid. No auto reset on CRC_DATA_32_BFLIP read.	R28
CRC_DATA	RW	For Write, must use a fixed width throughout the session. The CRC module supports lower 8-bit, or lower 16-bit, or full 32-bit data widths. For Read, LSB or CRC_DATA[0] is first bit on the wire. For Read, reset the CRC_DATA back to CRC_SEED state. Note: Firmware must add 1 to 2 NOPs after the last XOUT to the XIN. For CRC16, only CRC_DATA[15-0] is valid.	R29

6.4.6.2.2.2 CRC Programming Model

The following steps are performed by the PRU firmware to use the CRC module:

Step1: Configuration (optional)

- Configure CRC type:
For CRC32 operation, set CRC32_ENABLE using XOUT instruction with the following parameters:
 - Device ID = 1
 - Base register = R25
 - Size = 1
- Update CRC_SEED, if required using XOUT with the following parameters:
 - Device ID = 1
 - Base register = R28
 - Size = 1 to 4

Step 2:

- Load new CRC data into R29
- Push CRC data to the CRC16/32 module using XOUT with the following parameters:
 - Device ID = 1
 - Base register = R29
 - Size = 1 to 4
- 1 or 2 NOPS
- Load the accumulated CRC result into the PRU using the XIN instruction with the following parameters:
 - Device ID = 1
 - Base register = R29
 - Size = 4

Repeat Step 2, numbers 1 and 2 for each new CRC data.

Note

When a session starts, the PRU firmware must use the same write data width throughout the session.

6.4.6.2.2.3 PRU and CRC16/32 Interface (R9:R2)

The PRU_ICSSG system implements a new wide 32-Bytes data path. The firmware can perform one XOUT of 32-Bytes, the hardware will feed the CRC16/32 4-Bytes at a time. This will take 20 clock cycles for CRC16 and 12 clock cycles for CRC32 for a 32-Bytes XOUT.

Table 6-430. PRU Register to XFR Mapping

PRU Register	XFR ID	Domain/Function	Description
R9:R2 Data	1	Data	XOUT Only 1-Byte to 32-Bytes in size